LAB - 3.

1. Round Robin.

```
# include <stdio.h>
# include <stdbool.h>

struct Process {
        int id, AT, BT, RT, CT, WT, TAT;
        bool inQueue;
};
void round_Robin(struct process P[], int n, int TQ){
    int time=0, comp=0;
    int q[100], f=0, r=0;
    float totalTAT=0, totalWT=0;
    int ganttIdx=0;
    int ganttPID[100];
    int ganttTime[100];
    for (int i=0; i<n; i++){
        P[i].RT = P[i].BT;
        P[i].inQueue = false;
    }
    ganttTime[0]=0;
    while(comp ~~total~~ < n){
        for(int i=0; i<n; i++){
            if(P[i].AT <= time && !P[i].inQueue && P[i].RT>0){
            queue[rear++]=i;
            P[i].inQueue = true;
            }
        }
        if(front == rear){
            time++;
            continue;
        }
```

```
int idx = q[f++];
ganttPID[ganttIdx] = p[idx].id;
if (p[idx].RT > TQ){
    time += TQ;
    p[idx].RT -= TQ;
}else{
    time += p[idx].RT;
    p[idx].RT = 0;
    p[idx].CT = time;
    p[idx].TAT = p[idx].CT - p[idx].AT;
    p[idx].WT = p[idx].TAT - p[idx].BT;
    totalTAT += p[idx].TAT;
    totalWT += p[idx].WT;
    completed++;
}
ganttIdx++;
ganttTime[ganttIdx] = time;

for(int i=0; i<n; i++){
    if (p[i].AT<= time && !p[i].inQueue && p[i].RT>0){
        q[r++] = i;
        p[i].inQueue = true;
    }
}
if (p[idx].RT>0){
    queue[rear++] = idx;
}
}
printf("\n process\t AT\t BT\t TAT\t WT\n");
for(int i=0; i<n; i++){
    printf("P %d\t %d \t %d \t %d \t %d \n", p[i].id, p[i].AT,
    p[i].BT, p[i].CT, p[i].TAT, p[i].WT);
}
```

```
printf("\n gantt chart:\n");
for(int i=0; i<ganttIdx; i++) printf("-----");
printf("\n|");
for (int i=0; i<ganttIdx; i++) printf("P%d|", gantt[i]
printf("\n");
for(i=0; i<=ganttIdx; i++) printf("-----");
printf("\n%d", ganttTime[0]);
for(int i=1; i<=ganttIdx; i++) printf("  %d", ganttTime
printf("Enter time quantum:");
scanf("%d", &TQ);
printf("%d", ganttTime[i]);
printf("\n\n Average Tat: %.2f\n", totalTAT /n);
printf("\n Average wt: %.2f\n", totalWT /n);
}
int main()
int n, TQ;
printf("Enter number of process:");
scanf("%d", &n);
struct Process p[n];
for(int i=0; i<n; i++){
p[i].id = i+1;
printf("P%d AT:", i+1);
scanf("%d", &p[id].AT);
printf("P%d BT:", i+1);
scanf("%d", &P[i].BT);
}
Printf("Enter time Quantum");
Scanf("%d", &TQ);
round_robin(P, n, TQ);
return 0;
}
```

o/p : Enter number of processes : 4

P1 AT : 0

P1 BT : 5

1

4

2

2

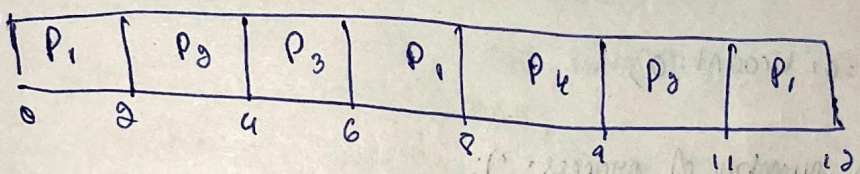
4

1

Enter TQ : 2

Process	AT	BT	CT	TAT	WT
P_1	0	5	12	12	7
P_2	1	4	11	10	6
P_3	2	2	6	4	2
P_4	4	1	9	5	4

Gantt chart:



Average Tat : 7.75

Average Wt : 4.75

2. Multilevel queue Scheduling

```
# include <stdio.h>

Struct process {
    int id, AT, BT, type, CT, TAT, WT);
};
void sort( Struct process P[], int n){
    Struct process temp;
    for (int i=0; i<n; i++){
     for (int j=0; j<n-i-1; j++){
       if P[j].AT > P[j+1].AT){
          temp = P[j];
          P[j] = P[j+1];
          P[j+1] = temp;
       }
     }
}
int main(){
 int n, scount=0, ucount=0;
 int time=0;
 Printf ("Enter number of process: ");
 scanf ("%d", &n);
 Struct process sys[n], user[n];

 for (int i=0; i<n; i++){
    Struct Process temp;
    temp.id = i+1;
    Printf (" P%d AT, BT, & type (system=0, user=1): ", i+1);
    Scanf ("%d %d %d", & temp.AT, & temp.BT, & temp.type);
    if (temp.type == 0){
       Sys[scount++] = temp; }
    else {
      user[ucount++] = temp; }
 }
 Sort (sys, scount);
 Sort (user, ucount);
```

```
for (int i=0; i< s(ount) ; i++
    if(time < Sys[i].AT) time = Sys[i].AT;
    time + = Sys[i].BT;
    Sys[i].CT = time;
    Sys[i].TAT = Sys[i].CT - Sys[i].AT;
    Sys[i].WT = Sys[i].TAT - Sys[i].BT;
    totTAT + = Sys[i].Tat;
    totWt + = Sys[i].Wt;
}
for (int i=0; i< u count; i++){
    if (time < user[i].AT) time = user[i].AT;
    time + = user[i].BT;
    User [i].CT = time;
    user [i].TAT = user[i].CT - user[i].AT;
    user [i].WT = user[i].TAT - user[i].BT;
    tot Tat + = user[i].TAT;
    tot wt + = user[i].WT;
}
printf("\n PID \t type \t AT \t BT BT \t CT \t TAT \t WT \n");
for(int i=0; i< S(ount ; i++)
    printf("P %d \t Sys \t %d \t %d \t %d \t %d \t %d \t",
    Sys [i].id, Sys [i].AT, Sys [i].BT, Sys[i].CT, Sys [i].TAT,
      Sys [i] WT);
for (int i=0; i<n(ount; i++)
    printf("P %d \t user \t %d \t %d \t %d \t %d \t %d \n",
    user[i].id, user[i].AT, user [i].BT, user [i]. CT,
    user [i]. TAT, user[i].WT);
printf ("\n gantt chart: \n");
for (int i=0; i< s(ount + u(ount ; i++)
    printf ("- --- ");
printf ("\n \n");
```

```
for (int i=0 ; i<sCount ; i++)
    printf(" %d ", sys[i].id);
for (int i=0; i<uCount ; i++);
    printf(" %d ", user[i].id);
printf("\n");
for (int i=0; i<sCount+ uCount ; i++)
    printf("----");
printf("\n");
for (int i=0; i<sCount ; i++)
    printf("----");
printf("\n AvgTat : %.2f", totTat \n);
printf("\n Avgwt : %.2f", totwt \n);
return 0;
}
```

output:

Enter number of processes: 4

P_1, AT, BT, & type (0 = system, 1 = user): 2

3
1
→ 4
2
0
→ 6
7
1
→ 3
4
0

PID	type	AT	BT	CT	TAT	WT
P_4	Sys	3	4	7	4	0
P_2	Sys	4	2	9	5	3
P_1	user	2	3	12	10	7
P_3	user	6	7	17	13	6

Gantt chart:

P_4	P_2	P_1	P_3

0 7 9 12 19

Avg Tat : 8.00

Avg wt : 4.00

17/3/26